Approximating π using Monte Carlo Simulation

James Chou

May 2026

Contents

1 Project Overview

Monte Carlo methods are computational algorithms that use repeated random sampling to model complex systems or obtain numerical approximations that would have been difficult to analytically compute.

Monte Carlo methods are especially useful for:

- Numerical integration
- High-dimensional problems
- Statistical physics
- Quantitative finance
- Stochastic optimisation.

In this project, I implemented Monte Carlo simulation to approximate the value of π.

The fundamental idea of my project is as follows:

- Randomly generate points all contained within a square of side length 2
- Consider a unit circle ($r = 1$) inscribed inside the square
- Determine whether each point lies inside the unit circle
- Use geometric probability to estimate π

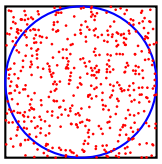


Figure 1: Visualisation of the concept used

We have

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi r^2}{(2)^2} = \frac{\pi}{4}$$

Since all points are contained within the square, we obtain

$$\pi \approx 4 \times \frac{\text{Number of points inside circle}}{\text{Total generated points}}.$$

2 Initial Basic Implementation

```python
import random

def estimate_pi(num_points):
    inside_circle = 0

    for _ in range(num_points):
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)

        distance_squared = x**2 + y**2

        if distance_squared <= 1:
            inside_circle += 1

    pi_estimate = 4 * inside_circle / num_points

    return pi_estimate

if __name__ == "__main__":
    num_points = 100000

    estimated_pi = estimate_pi(num_points)

    print(f"Estimated pi: {estimated_pi}")
```

Listing 1: basicversion.py

2.1 Explaining the Initial Algorithm

Step 1: Generate Random Points

Each point (x, y) is sampled uniformly from $[-1, 1] \times [-1, 1]$, which forms a square.

Step 2: Determining whether a point lies in circle or not

The equation of the unit circle is $x^2 + y^2 = 1$. So we can say that

$$\text{Point lies inside the circle} \iff x^2 + y^2 \leq 1$$

Step 3: Compute ratio

Let

- N = total generated points,
- M = points inside circle.

Then we have

$$\frac{M}{N} \approx \frac{\pi}{4}$$

$$\pi \approx 4 \cdot \frac{M}{N}$$

2.2 Weaknesses of my Initial Code

The first implementation uses Python loops and scalar arithmetic. It works correctly but has several computationalinefficiencies:

Issue	**Explanation**
Python loop overhead	Scalar iteration is relatively slow
Repeated conditional checks	Branching occurs for every point
Lack of vectorization	No use of optimized numerical (numpy)
Poor scalability	Runtime increases significantly for large simulations

Table 1: Weaknesses of Initial Version

3 Refined Implementation

I optimised the initial code by using NumPy vectorisation.

Advantages:

- Improved execution speed
- Cleaner code
- Efficient memory access

```python
import numpy as np
import time

def estimate_pi_vectorized(num_points, seed=42):

    np.random.seed(seed)

    start_time = time.time()

    x = np.random.uniform(-1, 1, num_points)
    y = np.random.uniform(-1, 1, num_points)

    distances = x**2 + y**2

    inside_circle = np.sum(distances <= 1)

    pi_estimate = 4 * inside_circle / num_points

    runtime = time.time() - start_time

    return pi_estimate, runtime

if __name__ == '__main__':
```

```
    num_points = 10_000_000

    estimate, runtime = estimate_pi_vectorized(num_points)

    print(f"Estimated pi: {estimate}")
    print(f"Runtime: {runtime:.4f} seconds")
```

Listing 2: optimisedversion.py

3.1 Improvements Implemented

3.1.1 Vectorisation

My initial code iterated point-by-point:

```
for _ in range(num_points):
```

Instead, I generated arrays simultaneously to delegate computation to optimised C implementations.:

```
x = np.random.uniform(-1, 1, num_points)
```

3.1.2 Faster Distance Computation

In my original code, I had this line being performed repeatedly for each point:

```
distance_squared = x**2 + y**2
```

Now, I made this line to be computed for entire arrays simultaneously:

```
distances = x**2 + y**2
```

3.1.3 Boolean Masking

In my original code, I used repeated conditionals (If statement):

```
if distance_squared <= 1:
```

Instead I used this, which produces a Boolean array efficiently.:

```
distances <= 1
```

3.1.4 Runtime Benchmarking

I added timing measurements to compare implementations quantitatively:

```
start_time = time.time()
```

3.2 Computational Complexity Analysis

3.2.1 Time Complexity

Time complexity measures how the amount of work performed by an algorithm grows as the input size N increases, focusing on asymptotic growth rather than actual execution times (refer to Section 4.1.1).

For the basic Monte Carlo implementation (basicversion.py), the main loop executes once for every generated sample:

```
for _ in range(num_points):
```

Each iteration performs a constant number of operations:

- generating two random numbers,
- computing $x^2 + y^2$,

- performing a comparison.

Consequently, the total number of operations is proportional to N and has linear time complexity $O(N)$.

Similarly, the vectorised NumPy implementation generates and processes arrays of length N. Every sample must be generated and checked exactly once, so the algorithm performs exactly N operations. Therefore the overall complexity remains $O(N)$.

Thus, both implementations have the same asymptotic time complexity $O(N)$ (it takes a constant factor of time proportional to the size of the input size). However, the optimised implementation substantially reduces constant factors due to the use of NumPy vectorisation.

3.2.2 Space Complexity

Initial Algorithm: The basic implementation stores only a few variables.

$$\implies O(1) \text{ space (constant space).}$$

- The algorithm requires a fixed amount of memory regardless of input size.

Refined Algorithm: The vectorised implementation stores arrays of length N.

$$\implies O(N) \text{ space (linear space)}$$

- The memory usage of the algorithm grows linearly with the input size, due to $O(N)$ space complexity.

So when using the refined algorithm, there is a tradeoff: faster execution but higher memory usage.

3.3 Visualisation Component

```python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

num_points = 5000

x = np.random.uniform(-1,1,num_points)
y = np.random.uniform(-1,1,num_points)

inside = x**2 + y**2 <= 1

plt.figure(figsize=(8,8))

plt.scatter(x[inside], y[inside], s=5)
plt.scatter(x[~inside], y[~inside], s=5)

circle = plt.Circle((0,0), 1, fill=False)

plt.gca().add_patch(circle)

plt.gca().set_aspect('equal')

plt.show()
```

Listing 3: visualisation.py

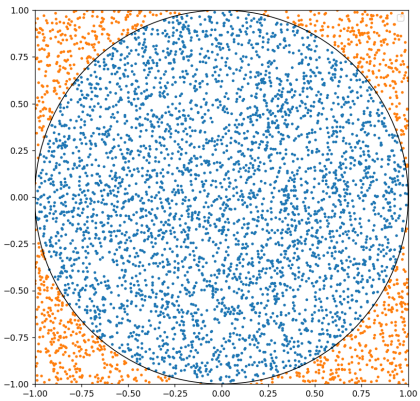
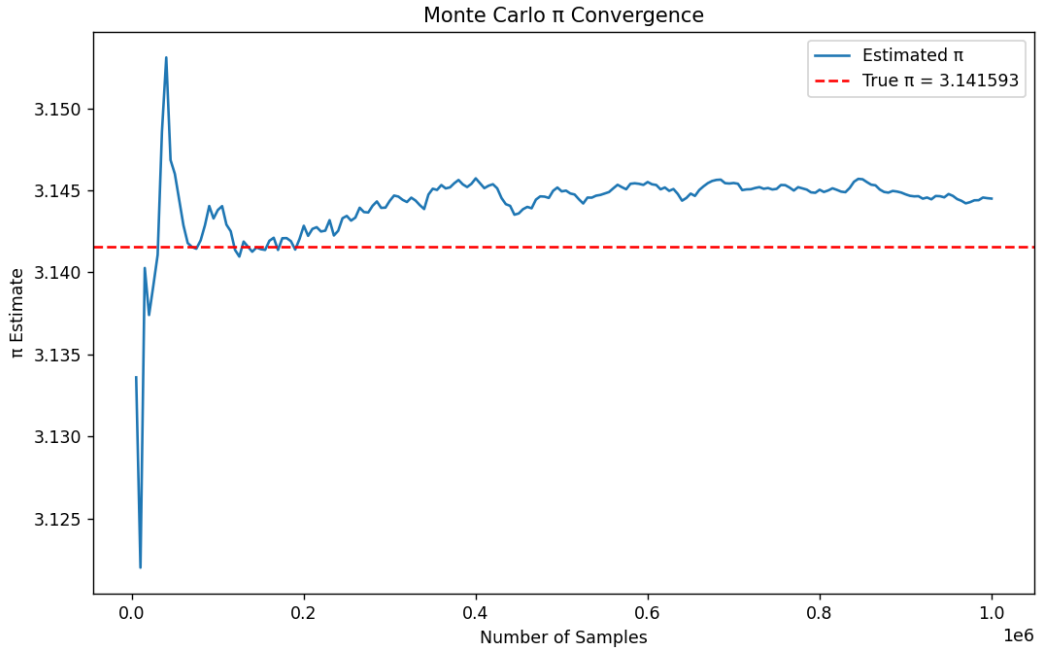


Figure 2: Visualising this Monte Carlo simulation

3.4 Convergence Analysis

```python
import numpy as np
import matplotlib.pyplot as plt

# parameters
TOTAL_POINTS = 1_000_000
BATCH_SIZE = 5000
SEED = 42

# random seed
np.random.seed(SEED)

# storage variables
inside_circle = 0
sample_counts = []
pi_estimates = []
errors = []

#plot setup
plt.ion()
fig, ax = plt.subplots(figsize=(10,6))
line_estimate, = ax.plot([], [], label='Estimated pi')
line_true = ax.axhline(
 np.pi,
 color='red',
 linestyle='--',
 label=f'True pi = {np.pi:.6f}'
)
ax.set_xlabel('Number of Samples')
ax.set_ylabel('pi Estimate')
ax.set_title('Monte Carlo Pi Convergence')
ax.legend()

# monte carlo simulation
for current_N in range(BATCH_SIZE,
 TOTAL_POINTS + 1,
 BATCH_SIZE):
  # Generate batch of random points
 x = np.random.uniform(-1, 1, BATCH_SIZE)
 y = np.random.uniform(-1, 1, BATCH_SIZE)
 # Count points inside circle
 inside_circle += np.sum(x**2 + y**2 <= 1)
 # Compute approximation
 approx_pi = 4 * inside_circle / current_N
 # Relative percentage error
 error = (
 100 * abs(approx_pi - np.pi) / np.pi
 )
 # Store values
 sample_counts.append(current_N)
 pi_estimates.append(approx_pi)
 errors.append(error)
 # Console monitoring
 print(
 f"N = {current_N:<10}"
 f"Approx pi = {approx_pi:.8f} "
 f"Error = {error:.6f}%",
 end='\r'
 )
 # Update graph
 line_estimate.set_data(
 sample_counts,
 pi_estimates
 )
 ax.relim()
 ax.autoscale_view()
 plt.pause(0.01)

# final display
plt.ioff()
plt.show()
```

Listing 4: convergenceanalysis.py



Continuous output that updates after each new sample point:

```
N = 1000000    Approx  = 3.14449600    Error = 0.092416%
```

The live convergence visualisation code combines:

- Vectorised batch sampling
- Live convergence plotting of our π estimate
- Continuous error monitoring for each N

Batch sampling allowed me to generate 5000 points at once and process them simultaneously, occasionally updating the graph after every processed batch:

```
BATCH_SIZE = 5000
```

$\implies$ This reduces Python overhead.

I used random seed, which generates pseudo-random numbers using deterministic algorithms. This ensures reproducibility and due to fixed randomness, we have:

- Repeatability of experiments,
- Verifiability of results,
- Fairness when benchmarking.

3.4.1 Mathematical Interpretation of Convergence

I will formalise the Monte Carlo estimator carefully.

For each randomly generated point, define the indicator random variable

$$X_i = \begin{cases} 1 & \text{if point lies inside circle,} \\ 0 & \text{otherwise.} \end{cases}$$

So each X_i is a Bernoulli random variable, where $X_i = 1$ represents a success and $X_i = 0$ a failure.

We now compute the probability of success:

A point is sampled uniformly from the square $[-1, 1] \times [-1, 1]$. The probability that a point lands inside the circle is

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi}{4}$$

$\therefore$ We have $\mathbb{P}(X_i = 1) = \frac{\pi}{4}$

We can now define $X_i \sim \text{Bernoulli}(\frac{\pi}{4})$.

Now suppose that we generate N points. Then we have

$$M = \sum_{i=1}^{N} X_i$$

where M is the number of points inside the circle.

So our Monte Carlo estimator is

$$\begin{aligned} \hat{\pi}_N &= 4 \cdot \frac{1}{N} \sum_{i=1}^{N} X_i \\ &= 4\,\overline{X}_N \end{aligned}$$

where $\overline{X}_N = \sum_{i=1}^{N} X_i$ is the sample mean.

We can observe that the Monte Carlo estimator is a scaled sample average.

We now invoke the **Weak Law of Large Numbers**, which I will first formally state:

Theorem (Weak Law of Large Numbers)**.** *If $X_1, X_2, \ldots$ are i.i.d. random variables with finite expectation μ, then*

$$\overline{X}_N \to \mu.$$

So by the Weak Law of Large Numbers, the sample average converges to the true mean.

We have $\mathbb{E}[]X_i = \frac{\pi}{4}$, so $\overline{X}_N \to \frac{\pi}{4}$. Multiplying both sides by 4, we have

$$\begin{aligned} 4\,\overline{X}_N &\to \pi \\ \hat{\pi}_N &\to \pi \end{aligned}$$

$\therefore$ As N increases,

$$\hat{\pi}_N \to \pi.$$

This explains why increasing the number of samples improves the approximation of π.

3.4.2 Error Behaviour

We compute the variance:

Since $X_i \sim \text{Bernoulli}(\frac{\pi}{4})$, the variance is

$$\text{Var}(X_i) = p(1-p)$$

with $p = \frac{\pi}{4}$. Therefore we have

$$\text{Var}(X_i) = \tfrac{\pi}{4}(1 - \tfrac{\pi}{4}).$$

I will now state the Central Limit Theorem:

Theorem (Central Limit Theorem)**.** *Suppose that X_1, X_2, X_N are i.i.d. random variables, satisfying $\mathbb{E}[X_i] = \mu < \infty$ and $Var(X_i) = \sigma^2 < \infty$. We have the sample mean*

$$\overline{X}_N = \frac{X_1 + X_2 + \cdots + X_N}{N}.$$

Then the normalised random variable

$$Z_N = \frac{\overline{X}_N - \mu}{\sigma/\sqrt{N}} = \frac{\sqrt{N}(\overline{X}_N - \mu)}{\sigma}$$

converges in distribution to the standard normal random variable as $N \to \infty$:

$$\frac{\sqrt{N}(\overline{X}_N - \mu)}{\sigma} \to \mathcal{N}(0,1)\,.$$

We now multiply both sides of the CLT result by σ to get

$$\sqrt{N}(\overline{X}_N - \mu) \to \mathcal{N}(0, \sigma^2)$$

since scaling a normal random variable by a constant will scale the variance by the square of the constant.

We now apply this to our Monte Carlo estimator:

$$\sqrt{N}\left(\overline{X}_N - \frac{\pi}{4}\right) \to \mathcal{N}\left(0, \frac{\pi}{4}\left(1 - \frac{\pi}{4}\right)\right)$$
$$\sqrt{N}\left(\hat{\pi}_N - \pi\right) \to \mathcal{N}\left(0, 4^2 \cdot \frac{\pi}{4}\left(1 - \frac{\pi}{4}\right)\right)$$
$$\sqrt{N}\left(\hat{\pi}_N - \pi\right) \to \mathcal{N}\left(0, 4\pi\left(1 - \frac{\pi}{4}\right)\right)$$

The CLT tells us that $\sqrt{N}(\hat{\pi}_N - \pi)$ has a limiting distribution with finite variance. Let

$$Y_N := \sqrt{N}(\hat{\pi}_N - \pi).$$

Since $Y_N \to \mathcal{N}(0, \sigma^2)$, we can say that for large N, Y_N behaves approximately like a normal random variable of constant size. Then we have

$$\hat{\pi}_N - \pi \approx \frac{Y_N}{\sqrt{N}}$$

where Y_N is a constant-sized random fluctuation. Therefore we have

$$\text{error magnitude} \propto \frac{1}{\sqrt{N}}.$$

$\therefore$ The CLT tells us that the fluctuations are approximately Gaussian and shrink like $\frac{1}{\sqrt{N}}$.

4 Benchmarking Script

```python
import time

from basicversion import estimate_pi
from optimisedversion import estimate_pi_vectorized

sample_sizes = [1000, 10000, 100000, 1000000]

print("Basic Version:")

for n in sample_sizes:

    start = time.time()

    estimate_pi(n)

    runtime = time.time() - start

    print(n, runtime)

print("Optimised Version:")

for n in sample_sizes:

    _, runtime = estimate_pi_vectorized(n)

    print(n, runtime)
```

Listing 5: benchmarking.py

An example of an output for my benchmarking code:

```
Basic Version:
1000 0.0
10000 0.0
100000 0.07797694206237793
1000000 0.7667508125305176
Optimized Version:
1000 0.0
10000 0.0
100000 0.0
1000000 0.03124213218688965
```

4.1 Evaluating the Benchmarking

4.1.1 Empirical Runtime Measurements

Empirical runtime measurements are obtained by executing the program and recording the time required to complete the computation.

It is noted that measurements vary slightly between runs. Such variation is expected because runtime is affected by factors external to the algorithm itself, which includes:

- operating system scheduling,
- background processes,
- CPU behaviour,
- memory allocation,
- timer precision,
- fluctuations in processor frequency.

Therefore benchmark results should be interpreted as approximate indicators rather than exact values.

A representative benchmark produced the following results:

Number of Samples	Basic Version (s)	Optimised Version (s)
1000	0.0000	0.0000
10000	0.0000	0.0000
100000	0.0780	0.0000
1000000	0.7668	0.0312

4.1.2 Evidence Supporting Linear Complexity

The benchmark cannot formally prove the complexity of the algorithm but it provides evidence which is consistent with previous analysis on time complexity in Section 3.2.1.

For the basic implementation, increasing the number of samples by a factor of 10 causes the runtime to increase by approximately the same order of magnitude:

$$\frac{0.7668}{0.0780} \approx 9.83.$$

This behaviour is characteristic of linear growth and is consistent with an $O(N)$ algorithm. Suppose that the implementation were quadratic, $O(N^2)$, increasing the input size by a factor of 10 would be expected to increase the runtime by approximately a factor of 100. However, this is clearly not observed in our benchmark results.

4.1.3 Why the Optimised Version is Faster

The benchmark results also demonstrate that the optimised implementation is substantially faster:

$$\frac{0.7668}{0.0312} \approx 24.6.$$

However this does not imply that the optimised implementation has a better asymptotic complexity, as both algorithms remain $O(N)$. Instead, vectorisation reduces the constant factor in the runtime model

$$T(N) = a + bN.$$

The basic implementation has a relatively large coefficient b due to Python loop overhead, whereas the vectorised implementation delegates most computations to highly optimised compiled code within NumPy, which results in a smaller coefficient.

Both implementations exhibit linear growth as N increases, but the optimised version performs the same amount of asymptotic work with a higher degree of efficiency.

4.1.4 Limitations of Current Benchmarking Script

There are some weaknesses that I identified in my current benchmark:

- Sample sizes are too small → several runtimes in the optimised version round to 0.0, making scaling analysis difficult
- Each benchmark is run only once → results are susceptible to random fluctuations from factors external to the algorithms

4.2 Refined Benchmarking Script

4.2.1 Improved Methodology of Benchmarking

I made a more rigorous benchmark which has the following properties:

- Using larger values of N
- Running each test multiple times
- Averaging the runtimes
- Storing the results
- Plotting the runtime curves

Instead of:

```
sample_sizes = [1000, 10000, 100000, 1000000]
```

I use:

```
sample_sizes = [10000, 100000, 1000000, 5000000, 10000000]
```

For each value of N:

- Run the algorithm 5 times,
- Compute the mean runtime,
- Record the result.

This substantially reduces noise.

4.2.2 Refined Benchmarking Script

```
import time
import numpy as np
import matplotlib.pyplot as plt

from basicversion import estimate_pi
from optimisedversion import estimate_pi_vectorized

# parameters
sample_sizes = [
    10_000,
    100_000,
    1_000_000,
    5_000_000,
    10_000_000
]

num_trials = 5


# storage
basic_runtimes = []
optimized_runtimes = []

# basic version
print("\nBenchmarking Basic Version")

for N in sample_sizes:

    trial_times = []

    for _ in range(num_trials):

        start = time.perf_counter()

        estimate_pi(N)

        end = time.perf_counter()

        trial_times.append(end - start)

    average_time = np.mean(trial_times)

    basic_runtimes.append(average_time)

```

```
    print(
        f"N = {N:<10}"
        f" Average Runtime = {average_time:.6f} s"
    )

# optimised version
print("\nBenchmarking Optimized Version")

for N in sample_sizes:

    trial_times = []

    for _ in range(num_trials):

        _, runtime = estimate_pi_vectorized(N)

        trial_times.append(runtime)

    average_time = np.mean(trial_times)

    optimized_runtimes.append(average_time)

    print(
        f"N = {N:<10}"
        f" Average Runtime = {average_time:.6f} s"
    )
```

Listing 6: refinedbenchmarking.py

Example output of the refined benchmarking code:

```
Benchmarking Basic Version
N = 10000      Average Runtime = 0.010512 s
N = 100000     Average Runtime = 0.101614 s
N = 1000000    Average Runtime = 0.975328 s
N = 5000000    Average Runtime = 4.994931 s
N = 10000000   Average Runtime = 9.864517 s

Benchmarking Optimized Version
N = 10000      Average Runtime = 0.001614 s
N = 100000     Average Runtime = 0.003133 s
N = 1000000    Average Runtime = 0.034468 s
N = 5000000    Average Runtime = 0.168641 s
N = 10000000   Average Runtime = 0.325813 s
```

I used:

```
time.perf_counter()
```

instead of:

```
time.time()
```

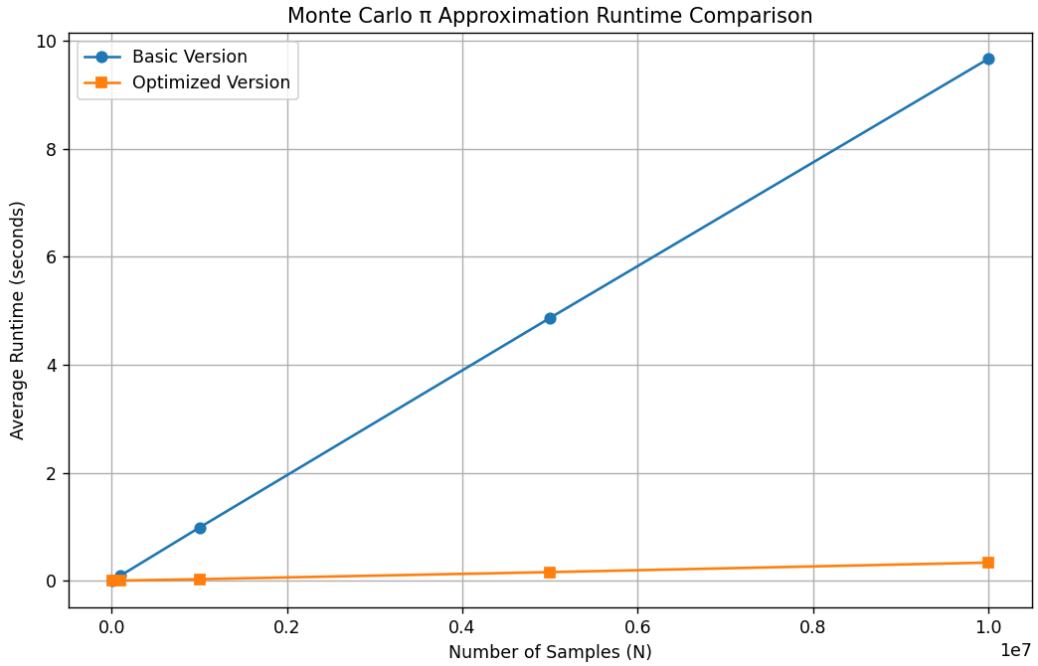
to provide higher-resolution timing measurements.

4.2.3 Runtime Comparison Plot

```
plt.figure(figsize=(10,6))

plt.plot(
    sample_sizes,
    basic_runtimes,
    marker='o',
    label='Basic Version'
)

plt.plot(
    sample_sizes,
    optimized_runtimes,
    marker='s',
    label='Optimized Version'
)

```

```
plt.xlabel('Number of Samples (N)')
plt.ylabel('Average Runtime (seconds)')

plt.title('Monte Carlo Pi Approximation Runtime Comparison')

plt.legend()

plt.grid(True)

plt.show()
```

Listing 7: runtimeplot.py



5 Extensions to the Initial Project

I thought of a computational and a mathematical extension, demonstrating both algorithmic optimisation and deeper numerical analysis.

5.1 Parallel Computing

5.1.1 Motivation

A key limitation of Monte Carlo methods is their slow convergence rate:

$$\text{Error} = O\left(\frac{1}{\sqrt{N}}\right).$$

To reduce the error, extremely large sample sizes are required.

We can utilise the fact that Monte Carlo simulations are naturally parallelizable because individual random samples require no information from one another, so each random point is generated independently.

5.1.2 Mathematical Justification

The estimator is

$$\hat{\pi}_N = \frac{4}{N}\sum_{i=1}^{N} X_i.$$

I can split the computational work across p processors.

Processor j $(1 \leq j \leq p)$computes

$$M_j = \sum_{i=1}^{N/p} X_i.$$

The total count becomes

$$M = M_1 + M_2 + \cdots + M_p.$$

Since each processor works independently, the estimator remains unchanged:

$$\hat{\pi} = \frac{4M}{N}.$$

5.1.3 Possible Python Implementation

Each CPU could simulate a subset of points using:

```
from multiprocessing import Pool
```

The multiprocessing package allows me to run code in parallel by leveraging multiple processors on my machine.

For sufficiently large N,

$$T_{\text{parallel}} \approx \frac{T_{\text{serial}}}{p}$$

where p is the number of cores.

Suppose we have a 10-core machine. Then the runtime may approach $\frac{1}{10}$ of the serial runtime.

5.2 Quasi-Monte Carlo Methods

5.2.1 Motivation

Monte Carlo relies on random sampling, where random points often cluster which leaves large gaps elsewhere. This contributes to variance.

I can replace pseudo-random points with constructed low-discrepancy sequences. Examples of these include:

- Halton sequences,
- Sobol sequences,
- Faure sequences.

These sequences produce much more even distribution.

5.2.2 Mathematical Justification

Standard Monte Carlo has error complexity

$$\text{Error} = O\left(\frac{1}{\sqrt{N}}\right)$$

Often, Quasi-Monte Carlo simulations achieve error complexity

$$\text{Error} = O\left(\frac{(\log N)^d}{N}\right)$$

in d dimensions.

5.2.3 Sobol Sampling

Instead of randomness, points are generated deterministically to fill the domain as evenly as possible.

Using SciPy:

```
from scipy.stats import qmc

sampler = qmc.Sobol(d=2)
points = sampler.random(n=N)
```

This produces points in $[0, 1]^2$ instead of each point being sampled from $[-1, 1]^2$ in random sampling.

I can then transform these points to obtain $[-1, 1]^2$:

```
points = 2*points - 1
```

This greatly reduces large gaps and clusters.

The Monte Carlo estimator error comes from irregular sampling. Random clustering means:

- some areas are oversampled,
- some areas are undersampled.

Sobol sequences minimise this effect.

Here $d = 2$, so asymptotically we have:

$$O\left(\frac{(\log N)^2}{N}\right) \leq O\left(\frac{1}{\sqrt{N}}\right)$$

6 Conclusion

In this project, I investigated the use of Monte Carlo simulation to approximate π. By uniformly sampling points within a square and determining the proportion that fell inside an inscribed unit circle, I derived an estimator for π from the ratio of the circle's area to the square's area.

The mathematical foundation of my method was established using indicator random variables. By defining

$$X_i = \begin{cases} 1 & \text{if the i-th point lies inside circle,} \\ 0 & \text{otherwise,} \end{cases}$$

it allowed me to express the Monte Carlo estimator as

$$\hat{\pi}_N = 4\left(\frac{1}{N}\sum_{i=1}^{N} X_i\right).$$

Since the probability of a randomly selected point lying inside the circle is $\frac{\pi}{4}$, the Weak Law of Large Numbers implies that

$$\hat{\pi}_N \to \pi.$$

Furthermore, the Central Limit Theorem showed that

$$\sqrt{N}(\hat{\pi}_N - \pi)$$

converges in distribution to a normal random variable. This implies that the typical magnitude of the estimation error decreases at the rate

$$O\left(\frac{1}{\sqrt{N}}\right).$$

This explains the convergence behaviour observed throughout the simulations and visualisations, where fluctuations become progressively smaller as sample size N increases.

From a computational perspective, I developed and compared two implementations. The initial implementation involved standard Python loops and scalar arithmetic, providing a simple and intuitive demonstration of the algorithm. Although the basic code exhibits linear time complexity, its reliance on Python-level iteration introduced substantial interpreter overhead and resulted in significantly longer runtimes for large N.

I addressed these limitations by developing a refined implementation using NumPy vectorisation. The improved code generated and processed large arrays of random samples simultaneously, which delegated the majority of computations to highly optimised compiled routines. I included additional refinements which included reproducibility through the use of fixed random seeds, and included real-time convergence visualisation with continuous error monitoring.

Benchmarking experiments allowed me to verify that the optimised version consistently achieved significantly lower runtimes due to reduced constant factors, despite both implementations possessing the same theoretical time complexity. Empirical measurements showed that runtime increased approximately proportionally to the number of samples, providing evidence consistent with the theoretical time complexity analysis.

The benchmark results also highlighted the distinction between asymptotic complexity and observed execution time. While vectorization does not alter the $O(N)$ growth rate, it dramatically improves practical efficiency by reducing the computational cost associated with each sample.

The live convergence visualisation further reinforced the probabilistic foundations of the method. As the number of samples increased, the estimated value of π exhibited progressively smaller fluctuations around the true value, providing an intuitive illustration of both the Law of Large Numbers and the Central Limit Theorem in action. The continuous monitoring of relative error also demonstrated the inherently slow convergence rate of Monte Carlo methods, where achieving an order-of-magnitude improvement in accuracy requires approximately one hundred times as many samples.

Since individual samples are independent, Monte Carlo methods are highly amenable to parallel execution. Distributing sample generation across multiple CPU cores can significantly reduce computation time without affecting estimator accuracy.

Sobol sampling exhibits lower discrepancy and therefore produces more uniform coverage of the sample space, resulting in smoother convergence and typically lower approximation error than ordinary Monte Carlo sampling.

Overall, I wanted this project to demonstrate how a simple geometric idea can be developed into a rigorous computational study combining probability theory, statistics, numerical simulation, algorithm analysis, and scientific programming. Beyond approximating π, the project provides insight into fundamental concepts that underpin modern computational mathematics, including stochastic simulation, statistical estimation, convergence theory, performance optimization, and empirical algorithm evaluation. The techniques explored here form the basis of many advanced applications in quantitative finance.